Basics of c++

Variables in C++ **2**
Types of Variables in C++ 4
1. Local Variable 4
2. Global Variable 4
3. Static Variable 4
4. Constant Variable 5
Operators in C++ 6
1. Arithmetic Operators 6
2. Relational (Comparison) Operators 6
3. Logical Operators 8
4. Assignment Operators 9
5. Bitwise Operators 10
6. Unary Operators 11
7. Ternary (Conditional) Operator 12
8. sizeof Operator 13
9. Comma Operator 13
10. Scope Resolution Operator (::) 13
11. Member Access Operators 14
12. Miscellaneous Operators 14
Summary of Operators **15**
Control Statements in C++ **16**
Types of Control Statements **16**
1. Selection (Decision-Making) Statements **16**
(a) if Statement 16
(b) if-else Statement 17
(c) Nested if Statement 18
(d) else-if Ladder 19
(e) switch Statement 20
Syntax 20
Example 21
Important Points of switch Statement 21
2. Iteration (Looping) Statements **22**
(a) while Loop 22
(b) do-while Loop 23
(c) for Loop 24
3. Jump (Branching) Statements **25**
(a) break Statement 25

(b) continue Statement..... 25
(c) goto Statement..... 26
(d) return Statement.....27

Variables in C++

Definition :

A **variable** is a named memory location used to store data. The value stored in a variable can be changed during program execution.

Syntax:

data_type variable_name;

or

data_type variable_name = value;

Examples
int age = 20;
float salary = 35000.50;
char grade = 'A';
double pi = 3.14159265;
bool isPassed = true;

Rules for Defining Variables in C++

Rule	Description	Example
1. Must start with a letter or underscore (_)	Variable name cannot begin with a digit.	`age`, `_count`
2. Cannot start with a number	Invalid if the first character is a digit.	`2num`
3. Can contain letters, digits, and underscore	No other special symbols are allowed.	`student1`, `emp_id`
4. No spaces allowed	Use underscore instead of space.	`student_name`

5. Cannot use C++ keywords	Reserved words cannot be variable names.	`int`, `class`, `while`
6. Variable names are case-sensitive	Uppercase and lowercase are treated differently.	`Age` and `age` are different variables.
7. Name should be meaningful	Improves code readability.	`totalMarks`

Valid Variable Names

- int age;
- float salary;
- char grade;
- int student1;
- double averageMarks;
- int _count;

Invalid Variable Names

- int 2age; // Starts with a number
- float total marks; // Contains space
- char class; // Keyword
- int roll-no; // Contains '-'
- float @price; // Contains '@'

Variable Declaration

A variable is **declared** before it is used.

- **int age;**
- **float salary;**
- **char grade;**

Variable Initialization

Initialization means assigning a value to a variable when it is declared.

- int age = 20;
- float marks = 95.5;
- char grade = 'A';

Variable Declaration and Assignment Separately

- int age; // Declaration
- age = 21; // Assignment

Types of Variables in C++

1. Local Variable

- Declared inside a function or block.
- Accessible only within that function/block.

```
#include<iostream>
using namespace std;

int main()
{
   int x = 10;   // Local variable
   cout << x;
   return 0;
}
```

2. Global Variable

- Declared outside all functions.
- Accessible throughout the program.

```
#include<iostream>
using namespace std;

int x = 100;

int main()
{
   cout << x;
   return 0;
}
```

3. Static Variable

- Retains its value between function calls.

```
#include<iostream>
using namespace std;

void fun()
{
   static int count = 0;
   count++;
   cout << count << endl;
}

int main()
{
   fun();
   fun();
   fun();
}
```

Output

1
2
3

4. Constant Variable

Declared using the `const` keyword.

const float PI = 3.14159;

Its value cannot be changed later.

Scope of Variables

Scope	Accessible In
Local	Inside the function/block only
Global	Entire program
Block	Inside `{}` only

Operators in C++

An **operator** is a symbol that performs an operation on one or more operands (variables or values).

1. Arithmetic Operators

Operator	Meaning	Example
+	Addition	`a+b`
-	Subtraction	`a-b`
*	Multiplication	`a*b`
/	Division	`a/b`
%	Modulus (remainder)	`a%b`
++	Increment	`a++`
--	Decrement	`a--`

Example

```
#include<iostream>
using namespace std;

int main()
{
   int a = 10, b = 3;

   cout << "Addition = " << a+b << endl;
   cout << "Subtraction = " << a-b << endl;
   cout << "Multiplication = " << a*b << endl;
   cout << "Division = " << a/b << endl;
   cout << "Modulus = " << a%b << endl;
}
```

2. Relational (Comparison) Operators

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

int a = 10, b = 20;

cout << (a > b); // 0
cout << (a < b); // 1

Example:

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 20;

    cout << (a == b) << endl;  // Equal to
    cout << (a != b) << endl;  // Not equal to
    cout << (a > b) << endl;   // Greater than
    cout << (a < b) << endl;   // Less than
    cout << (a >= b) << endl;  // Greater than or equal to
    cout << (a <= b) << endl;  // Less than or equal to

    return 0;
}
```

Output:

0
1

0
1
0
1

3. Logical Operators

Logical operators are used to **combine multiple conditions**. They return `true` (`1`) or `false` (`0`).

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

Example

int age = 22;

cout << (age > 18 && age < 30);

Example:

```
#include <iostream>
using namespace std;

int main()
{
    int a = 10;
    int b = 20;

    // Logical AND (&&)
    cout << (a < b && b > 15) << endl;

    // Logical OR (||)
    cout << (a > b || b > 15) << endl;

    // Logical NOT (!)
    cout << !(a < b) << endl;
```

```
    return 0;
}
```

Output:
1
1
0

4. Assignment Operators

Assignment operators are used to **assign or update values** stored in variables.

Operator	Example	Equivalent To
=	a = b	Assign value
+=	a += b	a = a + b
-=	a -= b	a = a - b
*=	a *= b	a = a * b
/=	a /= b	a = a / b
%=	a %= b	a = a % b

Example

```
#include <iostream>
using namespace std;

int main()
{
    int a = 10;

    // Simple assignment
    a = 20;
    cout << a << endl;

    // Add and assign
    a += 5;       // a = a + 5
```

```
    cout << a << endl;

    // Subtract and assign
    a -= 5;       // a = a - 5
    cout << a << endl;

    // Multiply and assign
    a *= 2;       // a = a * 2
    cout << a << endl;

    // Divide and assign
    a /= 2;       // a = a / 2
    cout << a << endl;

    // Modulus and assign
    a %= 3;       // a = a % 3
    cout << a << endl;

    return 0;
}
```

5. Bitwise Operators

Bitwise operators work **directly on the individual bits** of integer values.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	One's Complement
<<	Left Shift
>>	Right Shift

Example

```
int a = 5, b = 3;

cout << (a & b);
cout << (a | b);
cout << (a ^ b);
```

Program

```
#include <iostream>
using namespace std;

int main()
{
    int a = 10;   // Binary: 1010
    int b = 6;    // Binary: 0110

    // Bitwise AND (&)
    cout << "a & b = " << (a & b) << endl;

    // Bitwise OR (|)
    cout << "a | b = " << (a | b) << endl;

    // Bitwise XOR (^)
    cout << "a ^ b = " << (a ^ b) << endl;

    // Bitwise NOT (~)
    cout << "~a = " << (~a) << endl;

    // Left shift (<<)
    cout << "a << 1 = " << (a << 1) << endl;

    // Right shift (>>)
    cout << "a >> 1 = " << (a >> 1) << endl;

    return 0;
}
```

6. Unary Operators

Operate on a single operand.

Operator	Meaning
+	Unary Plus
-	Unary Minus
++	Increment
--	Decrement
!	Logical NOT
~	Bitwise NOT

Example

int a = 10;

++a;
--a;

7. Ternary (Conditional) Operator

The **ternary operator** is a short way of writing a simple `if-else` condition. It is also called the **conditional operator**.

Syntax

(condition) ? expression1 : expression2;

Example

int a = 10, b = 20;

int max = (a > b) ? a : b;
cout << max;

Example: Find the maximum number using conditional operator.

```
#include <iostream>      // Includes the input/output stream library
using namespace std;     // Allows us to use cout without writing std::cout

int main()
{
```

```
    int a = 10;          // Declare and initialize the first integer
    int b = 20;          // Declare and initialize the second integer

    // Ternary operator:
    // If a is greater than b, max gets the value of a.
    // Otherwise, max gets the value of b.
    int max = (a > b) ? a : b;

    cout << max;         // Display the maximum value

    return 0;            // Indicate successful program termination
}
```

8. sizeof Operator

Returns the size of a data type or variable in bytes.

cout << sizeof(int); // Usually 4
cout << sizeof(double); // Usually 8

9. Comma Operator

Evaluates multiple expressions from left to right.

int a;

a = (5, 10, 15);

cout << a;

Output

15

10. Scope Resolution Operator (::)

Used to access global variables, class members, or namespace members.

```cpp
#include<iostream>
using namespace std;

int x = 100;   // Global variable

int main()
{
   int x = 10;      // Local variable

   cout << x << endl;    // Local variable
   cout << ::x << endl;  // Global variable

   return 0;
}
```

Output

10
100

11. Member Access Operators

Operator	Purpose
.	Access object members
->	Access members through a pointer

Example

object.member;
pointer->member;

12. Miscellaneous Operators

Operator	Purpose
`new`	Allocates dynamic memory
`delete`	Frees dynamically allocated memory

`&`	Address-of operator
`*`	Pointer dereference

Example

```
int *ptr = new int;
*ptr = 50;

cout << *ptr;

delete ptr;
```

Summary of Operators

Operator Category	Operators
Arithmetic	`+ - * / % ++ --`
Relational	`== != > < >= <=`
Logical	`&& || !`
Assignment	`= += -= *= /= %=`
Bitwise	`& | ^ ~ << >>`
Unary	`+ - ++ -- ! ~`
Conditional	`?:`
Memory	`new delete`
Pointer	`* &`
Scope Resolution	`::`
Member Access	`. ->`
Size	`sizeof`
Comma	`,`

Control Statements in C++

Definition

Control statements are statements that control the flow of execution of a program. They determine the order in which statements are executed based on conditions, loops, or branching.

Purpose of Control Statements

- Make decisions based on conditions.
- Execute a block of code repeatedly.
- Transfer control from one part of the program to another.

Types of Control Statements

There are three main types of control statements in C++:

1. **Decision-Making Statements**
2. **Looping Statements**
3. **Jump Statements**

1. Selection (Decision-Making) Statements

These statements execute different blocks of code depending on whether a condition is **true** or **false**.

(a) if Statement

Syntax

```
if(condition)
{
   // Statements
}
```

Flow

- If the condition is **true**, the statements inside the `if` block execute.
- If the condition is **false**, the block is skipped.

Example

```
#include<iostream>
using namespace std;

int main()
{
   int age = 20;

   if(age >= 18)
   {
       cout << "Eligible for voting";
   }

   return 0;
}
```

Output

Eligible for voting

(b) if-else Statement

Used when there are two possible outcomes.

Syntax

```
if(condition)
{
   // True block
}
else
{
   // False block
}
```

Example

```
#include<iostream>
using namespace std;
```

```
int main()
{
   int number = 7;

   if(number % 2 == 0)
   {
       cout << "Even Number";
   }
   else
   {
       cout << "Odd Number";
   }

   return 0;
}
```

Output

Odd Number

(c) Nested if Statement

An `if` statement inside another `if` statement.

Example

```
#include<iostream>
using namespace std;

int main()
{
   int age = 25;
   bool license = true;

   if(age >= 18)
   {
       if(license)
       {
           cout << "You can drive.";
       }
   }
```

```
    return 0;
}
```

Output

You can drive.

(d) else-if Ladder

Used when multiple conditions need to be checked.

Syntax

```
if(condition1)
{
}
else if(condition2)
{
}
else if(condition3)
{
}
else
{
}
```

Example

```
#include <iostream>
using namespace std;

int main()
{
    // Store the student's marks
    int marks = 82;

    // Check if marks are 90 or above
    if(marks >= 90)
        cout << "Grade A";

    // If the above condition is false,
```

```
    // check if marks are 75 or above
    else if(marks >= 75)
        cout << "Grade B";

    // If the above conditions are false,
    // check if marks are 60 or above
    else if(marks >= 60)
        cout << "Grade C";

    // If none of the above conditions are true,
    // display "Fail"
    else
        cout << "Fail";

    return 0;   // Indicate successful program termination
}
```

Output
Grade B

(e) switch Statement

Used when one variable is compared against multiple constant values.

Syntax

```
switch(expression)
{
   case value1:
       statements;
       break;

   case value2:
       statements;
       break;

   default:
       statements;
}
```

Example

```
#include<iostream>
using namespace std;

int main()
{
   int day = 3;

   switch(day)
   {
       case 1:
           cout << "Monday";
           break;

       case 2:
           cout << "Tuesday";
           break;

       case 3:
           cout << "Wednesday";
           break;

       default:
           cout << "Invalid Day";
   }

   return 0;
}
```

Output

Wednesday

Important Points of `switch` Statement

- The **switch expression** is evaluated once.
- The expression must result in an **integral or enumeration type**, such as:
    - `int`
    - `char`
    - `short`
    - `long`
    - `enum`

- `float` and `double` **cannot be directly used** as the switch expression.
- Each `case` value must be a **constant integral expression**.
- Each `case` value must be **unique** within the switch.
- The `case` value must be compatible with the type of the switch expression.
- The `break` statement terminates the current `switch` block.
- If `break` is not used, execution **falls through** to the next `case`.
- The `default` block is **optional**.
- `default` executes when **none of the case values match**.
- The `default` block can be placed **anywhere** inside the switch, although it is normally written at the end.
- Multiple cases can execute the **same statement**.
- `case` labels cannot contain variables as their values.

2. Iteration (Looping) Statements

Loops execute a block of code repeatedly until a condition becomes false.

(a) while Loop

Syntax

```
while(condition)
{
   // Statements
}
```

Flow

- Checks the condition first.
- If true, executes the loop body.
- Repeats until the condition becomes false.

Example

```
#include<iostream>
using namespace std;

int main()
{
   int i = 1;
```

```
    while(i <= 5)
    {
        cout << i << " ";
        i++;
    }

    return 0;
}
```

Output

1 2 3 4 5

(b) do-while Loop

Syntax

```
do
{
    // Statements
}
while(condition);
```

Flow

- Executes the loop body first.
- Checks the condition afterward.
- Executes at least once, even if the condition is false.

Example

```
#include<iostream>
using namespace std;

int main()
{
    int i = 1;

    do
    {
        cout << i << " ";
        i++;
    }
```

```
    while(i <= 5);

    return 0;
}
```

Output

1 2 3 4 5

(c) for Loop

Used when the number of iterations is known.

Syntax

```
for(initialization; condition; increment)
{
    // Statements
}
```

Example

```
#include<iostream>
using namespace std;

int main()
{
    for(int i = 1; i <= 5; i++)
    {
        cout << i << " ";
    }

    return 0;
}
```

Output

1 2 3 4 5

Comparison of Loops

Feature	while	do-while	for

Condition Checked	Before execution	After execution	Before each iteration
Executes at least once	No	Yes	No
Best Used When	Number of iterations is unknown	Loop must execute at least once	Number of iterations is known

3. Jump (Branching) Statements

Jump statements transfer control from one point of the program to another.

(a) break Statement

Terminates the nearest loop or switch statement immediately.

Example

```
#include<iostream>
using namespace std;

int main()
{
   for(int i = 1; i <= 10; i++)
   {
       if(i == 6)
           break;

       cout << i << " ";
   }

   return 0;
}
```

Output

1 2 3 4 5

(b) continue Statement

Skips the remaining statements of the current iteration and starts the next iteration.

Example

```
#include<iostream>
using namespace std;

int main()
{
   for(int i = 1; i <= 5; i++)
   {
       if(i == 3)
           continue;

       cout << i << " ";
   }

   return 0;
}
```

Output

1 2 4 5

(c) goto Statement

Transfers control directly to a labeled statement.

Syntax

```
goto label;

// Statements

label:
   // Statements
```

Example

```
#include<iostream>
using namespace std;

int main()
{
```

```
    int i = 1;

start:
    cout << i << " ";
    i++;

    if(i <= 5)
        goto start;

    return 0;
}
```

Output

1 2 3 4 5

> **Note:** The use of `goto` is generally discouraged because it can make programs difficult to read and maintain.

(d) return Statement

Ends the execution of a function and optionally returns a value.

Example

```
#include<iostream>
using namespace std;

int main()
{
    cout << "Program Started";

    return 0;

    cout << "This will never execute";
}
```

Output

Program Started

Summary of Control Statements

Category	Statements	Purpose
conditional	`if`, `if-else`, `nested if`, `else-if`, `switch`	Decision making
looping	`while`, `do-while`, `for`	Repeating a block of code
Jump	`break`, `continue`, `goto`, `return`	Altering the normal program flow

Programming Questions:

1. The Greatest Number Among the Given Three Number
2. The Number Is Positive or Negative
3. Character Is Vowel or Consonant
4. A Character Is an Alphabet or Not
5. Uppercase, Lowercase, Special Character, or Digit
6. The Number Is Positive or Negative
7. The Number Is Even or Odd
8. Greatest of Two Numbers
9. Greatest Among Three Numbers
10. Leap Year
11. The Date Is Correct or Not
12. Voting Eligibility Checker
13. Find the maximum between two numbers.
14. Find the maximum between the three numbers.
15. Check whether a number is negative, positive or zero.
16. Check whether a number is divisible by 5 and 11 or not.
17. Find whether a number is even or odd.
18. Check whether a year is a leap year or not.
19. Check whether a character is an alphabet or not.
20. Input any alphabet and check whether it is vowel or consonant.
21. Input any character and check whether it is the alphabet, digit or special character.
22. Check whether a character is an uppercase or lowercase alphabet.
23. Input week number and print weekday
24. Input month number and print number of days in that month.
25. Count the total number of notes in a given amount.
26. Input angles of a triangle and check whether the triangle is valid or not.
27. Input all sides of a triangle and check whether the triangle is valid or not.
28. Check whether the triangle is an equilateral, isosceles or scalene triangle.
29. Calculate profit or loss